Library Management System

Design Document — SOLID Principles & Low-Level Design Patterns

---

A console-based C++ library management system, refactored from a single procedural file into a layered, SOLID-compliant design, then extended with Singleton, Factory, Observer, and State patterns to demonstrate Low-Level Design (LLD) skills.

1. Where This Project Came From

The original `main.cpp` was a single procedural file: global parallel arrays for students and books, fixed caps (`MAX_STUDENTS=20`, `MAX_BOOKS=15`), `char[]/strcpy` strings, and one `main()` function mixing login, menus, and business rules. No classes, no structs.

It went through two deliberate passes:

1. **SOLID Refactor** — same features, same numbers, split into classes with single responsibilities, wired through interfaces instead of globals. No new patterns beyond what the problem already needed (Repository, Strategy, and Facade fell out naturally).
2. **Pattern Showcase Extension** — explicitly requested to demonstrate LLD interview skills: Singleton, Factory, Observer, and State pattern added on top, plus new item types (DVD, Magazine) and new features (Return Item, View Issued Items) to give those patterns real work to do. This pass is intentionally more machinery than the app strictly needs — that is the stated goal, not an accident.

2. Layered Architecture

```
main.cpp                wiring only: builds repos, policy, observer, service, UI
      |
ConsoleUI                all cin/cout, menu loops, no business logic
      |  (talks only to LibraryService)
LibraryService             business logic / orchestration (the Facade)
      |            |                  |
IRepository<Student>  IItemRepository   IFeePolicy
(Student storage)    (LibraryItem storage)  (pricing rules)
                           |
                      LibraryItem (abstract)
                      +-- Book
                      +-- DVD
                      +-- Magazine
                           |
                       ItemState (Available / Issued)

NotificationCenter (global event bus, Singleton)
      |
IObserver  <-  ConsoleAuditObserver (prints [NOTICE] lines)
```

Nothing above `LibraryService` knows about repositories, item types, or the notification bus directly — `ConsoleUI` only calls `LibraryService` methods.

3. Class Responsibilities (SRP)

Class	File	Single Responsibility
Student	src/Student.*	Roll/name/balance + deposit/withdraw
LibraryItem, Book, DVD, Magazine	src/LibraryItem.*, Book.*, DVD.*, Magazine.*	Item identity + delegate availability to state
ItemState (Available/Issued)	src/ItemState.*	Availability transitions only

Class	File	Single Responsibility
ItemFactory	src/ItemFactory.*	Construct the right LibraryItem subtype
IRepository / InMemoryRepository	src/IRepository.h, InMemoryRepository.h	Store/find Student values
IItemRepository / InMemoryItemRepository	src/IItemRepository.h, InMemoryItemRepository.h	Own/find polymorphic LibraryItem pointers
IFeePolicy / StandardFeePolicy	src/FeePolicy.*	Pricing numbers only
Authenticator	src/Authenticator.*	Password check only
NotificationCenter	src/NotificationCenter.*	Pub/sub event bus only
IObserver / ConsoleAuditObserver	src/IObserver.h, ConsoleAuditObserver.*	React to an event (print it)
Console	src/Console.*	ANSI colors / header / divider helpers only
LibraryService	src/LibraryService.*	Orchestrates everything above (the Facade)
ConsoleUI	src/ConsoleUI.*	All terminal I/O and menu flow

4. SOLID Principles — Where and How

S — Single Responsibility

Each class changes for exactly one reason (see table above). `ItemState` doesn't know about students; `NotificationCenter` doesn't know about fees; `ConsoleUI` doesn't know about repositories.

O — Open/Closed

Adding a new item type = new `LibraryItem` subclass + one case in `ItemFactory::create` (`src/ItemFactory.cpp`) — `LibraryService` and `ConsoleUI` don't change. Adding a new notification channel = implement `IObserver`, call `subscribe()` — no change to any publisher. Adding a new pricing scheme = new `IFeePolicy` implementation, no change to `LibraryService`.

L — Liskov Substitution

`Book`, `DVD`, `Magazine` are fully interchangeable wherever `LibraryItem*` is used — `LibraryService::issueItem`, `editItem`, `ConsoleUI::viewAllItems`, etc. never downcast or switch on type; they call `describe()`, `issue()`, `returnItem()` polymorphically.

I — Interface Segregation

`IItemRepository` (4 methods), `IObserver` (1 method), `IRepository` (4 methods) — each interface is only what its caller needs. `LibraryService`'s public methods are grouped by caller (student operations vs admin operations) so `ConsoleUI` only calls what each menu needs.

D — Dependency Inversion

`LibraryService` depends on `IRepository&`, `IItemRepository&`, `const IFeePolicy&` — all injected through its constructor (`src/LibraryService.h`), never constructed internally. The one deliberate exception is the Singleton notification bus, described in section 6.

5. Design Patterns — Where and How

Repository

`src/IRepository.h / InMemoryRepository.h`, `src/IItemRepository.h / InMemoryItemRepository.h`

Abstracts storage behind `add`/`find`/`all`/`count`. `LibraryService` never touches a `std::vector` directly. Two separate interfaces exist because `IRepository` stores values by copy (fine for `Student`) while `IItemRepository` must own polymorphic `LibraryItem` objects via `std::unique_ptr` (a value-type template can't hold subclasses safely) — a genuine C++ constraint worth mentioning if asked "why two repository interfaces instead of one generic one?"

Strategy

`src/FeePolicy.h/.cpp`

`IFeePolicy` defines `accountOpeningCost()`, `securityDeposit()`, `issueFee()`, `minimumInitialDeposit()`. `LibraryService` asks the policy for numbers instead of hardcoding them, so pricing can be swapped by implementing a new policy class.

Facade

`src/LibraryService.h/.cpp`

`ConsoleUI` (and any future caller — tests, a GUI) talks to one `LibraryService` object instead of juggling two repositories, a fee policy, and the notification bus individually.

Factory

`src/ItemFactory.h/.cpp`

```
enum class ItemType { Book, Dvd, Magazine };
static std::unique_ptr<LibraryItem> create(
    ItemType type, title, author, catalogId, extraNumeric);
```

`LibraryService::addItem()` calls `ItemFactory::create(...)` instead of `new Book(...)` / `new DVD(...)` directly. `extraNumeric` is reused as runtime-minutes for DVD or issue-number for Magazine — a deliberate simplification over a full Builder, to keep the console prompt to one extra question. Caveat worth naming out loud: a plain switch-based factory still needs editing for a 4th item type — a self-registering factory (map of type to creation function) would close that OCP gap, but wasn't built here to avoid stacking patterns beyond what's being shown.

Observer

`src/IObserver.h`, `src/LibraryEvent.h`, `src/ConsoleAuditObserver.h/.cpp`

`IObserver::onEvent(const LibraryEvent&)` is the listener contract. `ConsoleAuditObserver` is the one concrete subscriber, printing `[NOTICE] ...` in yellow (`src/Console.h/.cpp` for the color helper). `LibraryService` publishes a `LibraryEvent` after: item added, item issued, item returned, account created, and balance dropping below $5 (`LOW_BALANCE_THRESHOLD` in

`src/LibraryService.h`).

Singleton

`src/NotificationCenter.h/.cpp`

```
static NotificationCenter& instance(); // function-local static, Meyer's singleton
```

Chosen as the home for Singleton deliberately instead of making `LibraryService` itself global — `LibraryService` stays constructor-injected and testable; `NotificationCenter` is the one place a shared global actually fits the Observer pattern's "subject everyone can subscribe to" shape. Copy constructor and assignment are deleted.

State

`src/ItemState.h/.cpp`, used from `src/LibraryItem.h/.cpp`

`LibraryItem` holds an `ItemState*` instead of a `bool available_`. `issue()`/`returnItem()` delegate to `state_->issue(*this)` / `state_->returnItem(*this)`. `AvailableState::issue()` transitions the item to `IssuedState`; `IssuedState::returnItem()` transitions it back. Both states are stateless, so each is a single shared instance (`AvailableState::instance()` / `IssuedState::instance()`, function-local statics) rather than allocated per item — the standard real-world detail for a State pattern with no per-state data.

6. Known, Deliberate Tradeoffs

Worth saying out loud in an interview — these are acknowledged design costs, not oversights:

Singleton reduces testability — `LibraryService` calls `NotificationCenter::instance()` directly rather than taking an `IObserver`-collection dependency through its constructor. That's the textbook criticism of Singleton: global state instead of an injected dependency. Accepted here specifically to demonstrate the pattern.

Factory needs one edit per new type — the plain switch-based `ItemFactory::create` must be edited for a 4th item type; a self-registering factory map would close that OCP gap.

No due dates / fines / account suspension — scoped out on purpose to keep the item lifecycle (State pattern) and notification flow (Observer) demonstrable without adding a clock abstraction or a second state machine. `ItemState` only models Available ↔ Issued.

`extraNumeric` in `ItemFactory::create` — a generic int standing in for "the one extra field this type needs" is a simplification over a Builder pattern per item type — reasonable at 3 item types, would stop being reasonable at a 4th or 5th with genuinely different extra fields.

7. Feature Walkthrough

Admin: Add item (Book/DVD/Magazine via Factory) → Edit item → View item status (polymorphic `describe()` + Available/Issued) → View enrolled students (sorted) → View one student's balance. Menu loops until 0 (logout) in `ConsoleUI::handleAdminMenu`.

Student: Create account (if roll unknown) → View balance → Deposit → Issue item (from available list) → Return item (from items currently held) → View issued items. Menu loops until 0 (logout) in `ConsoleUI::handleStudentMenu`.

Every issue/return/add/create/low-balance event prints a yellow [NOTICE] line via the Observer, independent of the menu flow that triggered it.

8. Build & Run

```
g++ -std=c++17 -Wall main.cpp src/Book.cpp src/DVD.cpp src/Magazine.cpp \
    src/LibraryItem.cpp src/ItemState.cpp src/ItemFactory.cpp \
    src/NotificationCenter.cpp src/ConsoleAuditObserver.cpp src/Console.cpp \
    src/Student.cpp src/FeePolicy.cpp src/Authenticator.cpp \
    src/LibraryService.cpp src/ConsoleUI.cpp -o library.exe
./library.exe
```

or simply `make`. Login password for both roles: `password`.